



第二章 数码与码制

周玲

ling.zhou@seu.edu.cn

内容



- 2.1 数制
- 2.2 算术运算
 - 2.2.1 二进制数的算术运算
 - 2.2.2 八进制数的算术运算
 - 2.2.3 十六进制数的算术运算
- 2.3 数制之间的转换
 - 2.3.1 R进制转换为十进制
 - 2.3.2 十进制转换为二进制
 - 2.3.3 二进制数与八进制数、十六进制数之间的相互转换
- 2.4 计算机中数的表示方法
 - 2.4.1 原码及其运算
 - 2.4.2 补码及其运算
 - 2.4.3 反码及其运算
- 2.5 计算机中的码
 - 2.5.1 码的概念
 - 2.5.2 数值编码
 - 2.5.3 字符码和其它码
 - 2.5.4 检错码和纠错码
- 本章小结

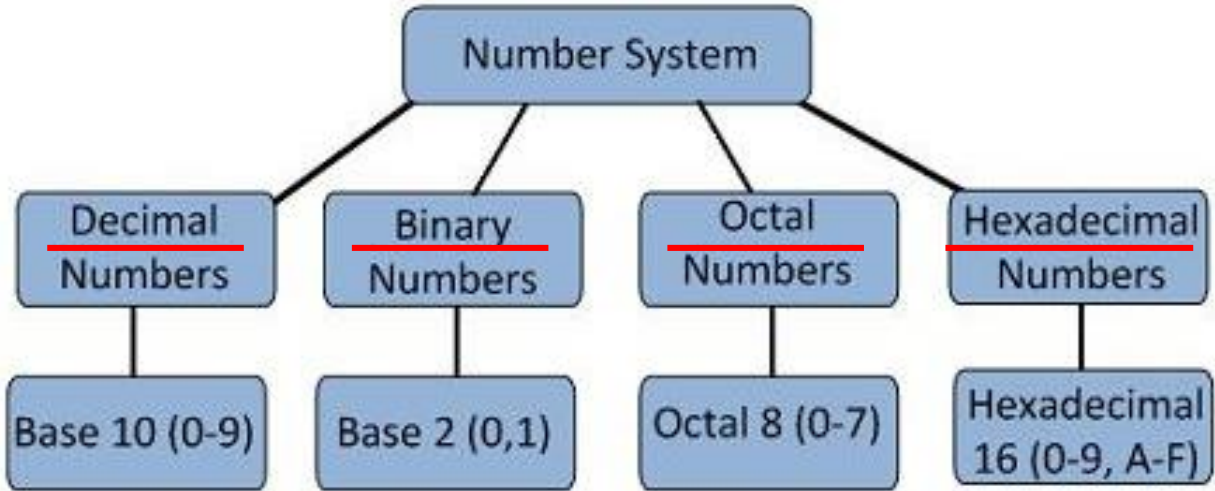
介绍

- 计算机中使用的是 二 进制数
 - 要表示为 机器数：原码、反码和补码。
- 为了简化 二 进制书写，计算机中还会使用 十六 进制和 八 进制

数制:计数进位制

常用进位计数制表示方法

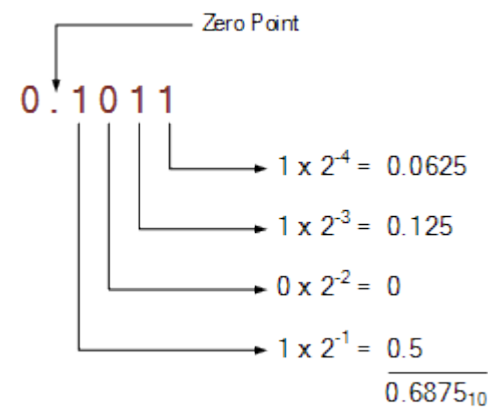
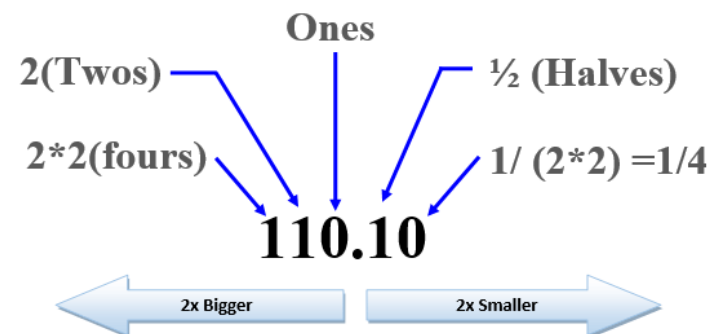
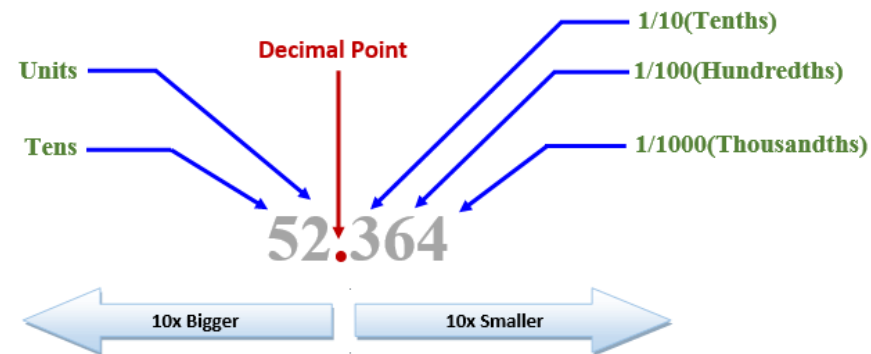
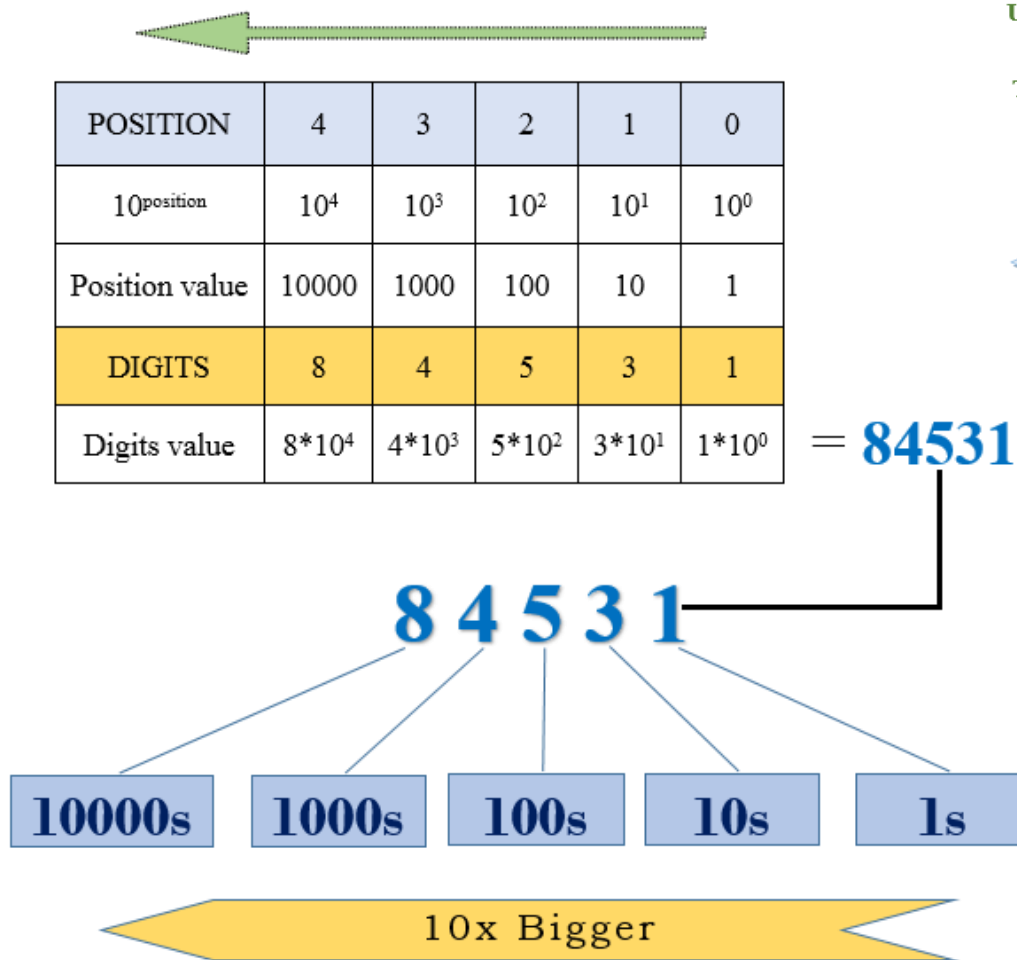
进位制	后缀	进位规则	基数R	权	数码 a_i
十进制	D或10	逢十进一	10	10^n	0,1,...,9
二进制	B或2	逢二进一	2	2^n	0,1
八进制	O或8	逢八进一	8	8^n	0,1,...,7
十六进制	H或16	逢十六进一	16	16^n	0,1,...,9,A,B,C,D,E,F



○ R进制

$$\begin{aligned}(N)_R &= a_{n-1}a_{n-2} \cdots a_1 \mathbf{a_0} a_{-1}a_{-2} \cdots a_{-m} \\ &= a_{n-1} \times R^{n-1} + a_{n-2} \times R^{n-2} + \cdots + a_1 \times R^1 + \mathbf{a_0 \times R^0} + a_{-1} \times R^{-1} + a_{-2} \times R^{-2} + \cdots + a_{-m} \times R^{-m} \\ &= \sum a_i \times R^i\end{aligned}$$

$$(123.45)_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0 + 4 \times 10^{-1} + 5 \times 10^{-2}$$



•不同进制数的对应关系:

十进制	二进制	八进制	十六进制
0	0	0	0
1	1	1	1
2	10	2	2
3	11	3	3
4	100	4	4
5	101	5	5
6	110	6	6
7	111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F
16	10000	20	10
17	10001	21	11
18	10010	22	12
...	...	...	...

内容



shutterstock.com • 1161804748

- 2.1 数制
- 2.2 算术运算
 - 2.2.1 二进制数的算术运算
 - 2.2.2 八进制数的算术运算
 - 2.2.3 十六进制数的算术运算
- 2.3 数制之间的转换
 - 2.3.1 R进制转换为十进制
 - 2.3.2 十进制转换为二进制
 - 2.3.3 二进制数与八进制数、十六进制数之间的相互转换
- 2.4 计算机中数的表示方法
 - 2.4.1 原码及其运算
 - 2.4.2 补码及其运算
 - 2.4.3 反码及其运算
- 2.5 计算机中的码
 - 2.5.1 码的概念
 - 2.5.2 数值编码
 - 2.5.3 字符码和其它码
 - 2.5.4 检错码和纠错码
- 本章小结

算术运算

- 对于任意进制数，首先要知道该进制数的一位数加法和乘法表，这样就可以采用与十进制数相同的方法，进行加、减、乘、除算术运算。

减法为加法反运算
除法为乘法反运算



+	1	2	3	4	5	6	7	8	9
1	2	3	4	5	6	7	8	9	10
2	3	4	5	6	7	8	9	10	11
3	4	5	6	7	8	9	10	11	12
4	5	6	7	8	9	10	11	12	13
5	6	7	8	9	10	11	12	13	14
6	7	8	9	10	11	12	13	14	15
7	8	9	10	11	12	13	14	15	16
8	9	10	11	12	13	14	15	16	17
9	10	11	12	13	14	15	16	17	18

Math Worksheets, Charts and Tables @ www.mathworksheets4kids.com

MULTIPLICATION TABLE

	1	2	3	4	5	6	7	8	9
1	1	2	3	4	5	6	7	8	9
2	2	4	6	8	10	12	14	16	18
3	3	6	9	12	15	18	21	24	27
4	4	8	12	16	20	24	28	32	36
5	5	10	15	20	25	30	35	40	45
6	6	12	18	24	30	36	42	48	54
7	7	14	21	28	35	42	49	56	63
8	8	16	24	32	40	48	56	64	72
9	9	18	27	36	45	54	63	72	81

二进制数的算术运算（加减）

+	0	1
0	0	1
1	1	10

-	0	1
0	0	1
1	1	0

借位
borrow 1

1286432168421

10011011

128+0+0+16+8+0+2+1

= 155

溢出
Overflow?

进位
Carried Values

01001110

A8A7A6A5A4A3A2A1

01001011

B8B7B6B5B4B3B2B1

二进制数的算术运算（乘除）

×	0	1
0	0	0
1	0	1

A ÷ B	
0 ÷ 1	0
1 ÷ 1	1

方法一

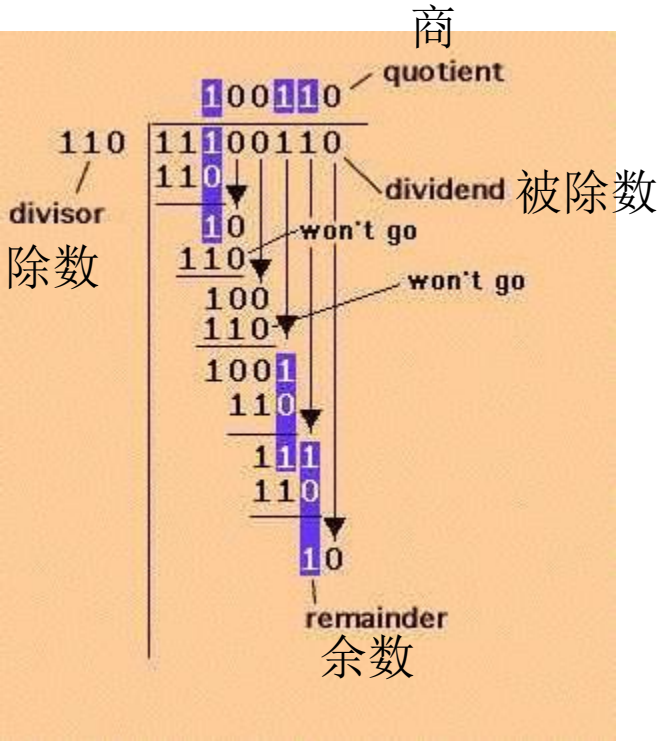
$$\begin{array}{r} 10101 \\ \times \quad 101 \\ \hline 10101 \\ 000000 \\ 1010100 \\ \hline 1101001 \end{array}$$

方法二

$$\begin{array}{r} 1101 \\ \times 1010 \\ \hline 0000 \\ 1101 \\ 0000 \\ 1101 \\ \hline 1000010 \end{array}$$

方法三

$$\begin{array}{r} 1011.01 \\ \times \quad 110.1 \\ \hline 101101 \\ 0 \\ \hline 101101 \\ 101101 \\ \hline 1001001.001 \end{array}$$



八进制数的算术运算



像十进制一样正常加减乘除，
然后考虑R进制

+	0	1	2	3	4	5	6	7
0	0	1	2	3	4	5	6	7
1	1	2	3	4	5	6	7	10
2	2	3	4	5	6	7	10	11
3	3	4	5	6	7	10	11	12
4	4	5	6	7	10	11	12	13
5	5	6	7	10	11	12	13	14
6	6	7	10	11	12	13	14	15
7	7	10	11	12	13	14	15	16

×	0	1	2	3	4	5	6	7
0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7
2	0	2	4	6	10	12	14	16
3	0	3	6	11	14	17	22	25
4	0	4	10	14	20	24	30	34
5	0	5	12	17	24	31	36	43
6	0	6	14	22	30	36	44	52
7	0	7	16	25	34	43	52	61

$$\begin{array}{r}
 \text{11 Carry 进位} \\
 456_8 \text{ Augend 被加数} \\
 + 123_8 \text{ Addend 加数} \\
 \hline
 601_8 \text{ Sum 和}
 \end{array}$$

$$\begin{array}{r}
 \text{11111 Carry} \\
 77714_8 \text{ Augend} \\
 + 76_8 \text{ Addend} \\
 \hline
 100012_8 \text{ Sum}
 \end{array}$$

$$\begin{array}{r}
 4167 \\
 \times 2503 \\
 \hline
 14545 \\
 0 \\
 25123 \\
 10356 \\
 \hline
 13105045
 \end{array}$$

十六进制数的算术运算

+	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
2	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
3	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
4	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
5	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
6	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
7	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
8	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
9	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
A	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
B	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
C	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
D	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
E	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
F	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E

Y

X

Sum

hex

$$\begin{array}{r}
 0011 \\
 11D7 \\
 + 16E \\
 \hline
 1345
 \end{array}$$

$$\begin{array}{r}
 1001 \text{ carry} \\
 8A5C \\
 + F39A \\
 \hline
 17DF6
 \end{array}$$

X \ Y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
2	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
3	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
4	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
5	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
6	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
7	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
8	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
9	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
A	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
B	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
C	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
D	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
E	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
F	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E

内容



shutterstock.com • 1161804748

- 2.1 数制
- 2.2 算术运算
 - 2.2.1 二进制数的算术运算
 - 2.2.2 八进制数的算术运算
 - 2.2.3 十六进制数的算术运算
- 2.3 数制之间的转换
 - 2.3.1 R进制转换为十进制
 - 2.3.2 十进制转换为二进制
 - 2.3.3 二进制数与八进制数、十六进制数之间的相互转换
- 2.4 计算机中数的表示方法
 - 2.4.1 原码及其运算
 - 2.4.2 补码及其运算
 - 2.4.3 反码及其运算
- 2.5 计算机中的码
 - 2.5.1 码的概念
 - 2.5.2 数值编码
 - 2.5.3 字符码和其它码
 - 2.5.4 检错码和纠错码
- 本章小结

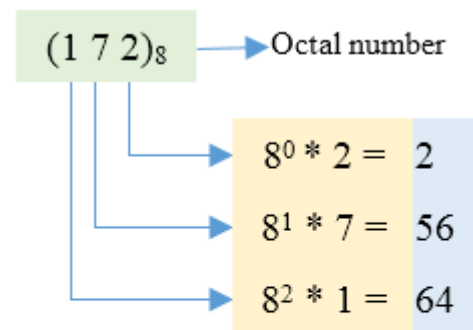
数制之间的转换

○ R 进制转换为十进制

- 只要按照 R 进制数的权位展开，再按照十进制的运算规则，将其各位数值相加就可以得到十进制数。
- $(10011011)_2 = (155)_{10}$

128	64	32	16	8	4	2	1
1	0	0	1	1	0	1	1
128 + 0 + 0 + 16 + 8 + 0 + 2 + 1							
= 155							

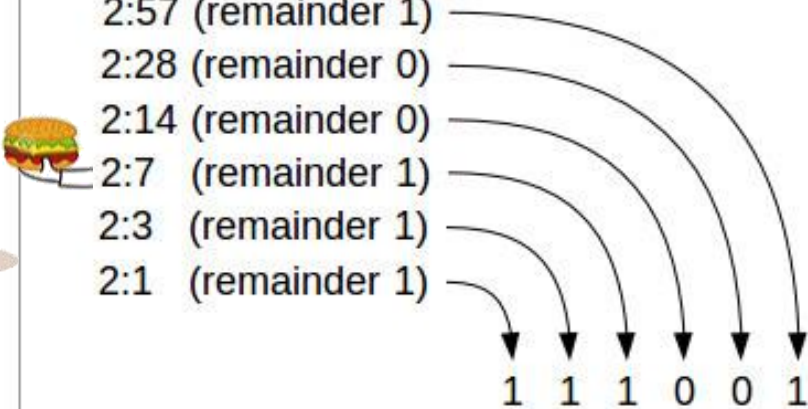
- $(172)_8 = (122)_{10}$



$$(172)_8 = (2 + 56 + 64)_{10}$$

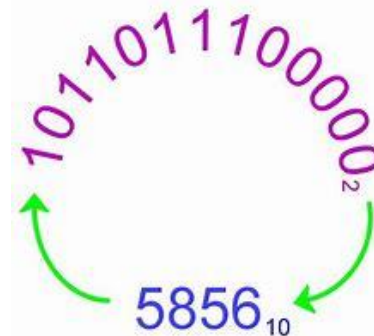
$$(172)_8 = (122)_{10}$$

Decimal to Binary Conversion

Convert decimal number 57 ₁₀ to binary	
Method 1 Descending Powers of Two and Subtraction	Method 2 Short Division by Two with Remainder
1. Write the power of two 凑 $2^0 = 1$ $2^1 = 2$ $2^2 = 4$ $2^3 = 8$ $2^4 = 16$ $2^5 = 32$ $2^6 = 64$ $2^7 = 128$ $2^8 = 256$  2. Write the number as a sum of powers of two $ \begin{aligned} 57 &= 32 + 25 \\ &= 32 + 16 + 9 \\ &= 32 + 16 + 8 + 1 \\ &= 1 \times 32 + 1 \times 16 + 1 \times 8 + 1 \times 1 \\ &= 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^0 \\ &= 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \end{aligned} $ 3. Extract the coefficients of the powers of two $57_{10} = 111001_2$	1. Divide continuously by two until we reach 0 $2:57$ (remainder 1) $2:28$ (remainder 0) $2:14$ (remainder 0) $2:7$ (remainder 1) $2:3$ (remainder 1) $2:1$ (remainder 1)  2. Write the remainders in reverse order (last is first) $57_{10} = 111001_2$

十进制转换为二进制

MSB – Most Significant Bit
LSB – Least Significant Bit



- 需要将十进制数分为整数部分和小数部分分别转换

整数部分:
除以2取余数, 逆序排列

Successive Division by 2

$$\begin{array}{r} 2 \overline{) 29} \\ 2 \overline{) 14} \\ 2 \overline{) 7} \\ 2 \overline{) 3} \\ 2 \overline{) 1} \\ 0 \end{array}$$

Remainders

1 LSB
0
1
1
1 MSB

Read the remainders from the bottom up

直到0才能结束

29 decimal = 11101 binary

小数部分
乘2取整数, 顺序排列

直到小数点后为0。

$(458.692)_{10} = (?)_2$

$.692 \times 2 = 1.384$	1 MSB
$.384 \times 2 = 0.768$	0
$.768 \times 2 = 1.536$	1
$.536 \times 2 = 1.072$	1 LSB

有的乘积小数点后一直不为0, 只能根据精度需要, 截取小数点后若干位来表达。

例如: 误差 $\varepsilon < 2^{-6}$, 则小数点后面取 位

ANOTHER EXAMPLE: DECIMAL TO BINARY

remainders

2	) 39	
2	) 19	----- 1
2	) 9	----- 1
2	) 4	----- 1
2	) 2	----- 0
2	) 1	----- 0
	0	----- 1



$$39_{10} = 100111_2$$

直到0才能结束



二进制数与八进制数、十六进制数相互转换

1位八进制数与
3位二进制数
是一一对应的

Octal Symbol	Binary equivalent
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

$$2^3 = 8$$
$$2^4 = 16$$

1位十六进制数
与4位二进制数
是一一对应的

Decimal	Binary	Hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

转换实例

○ 10101111.0001011011B → Octal

Binary 010 101 111. 000 101 101 100
Octal 2 5 7 . 0 5 5 4

○ 10101111.0001011011B → Hexadecimal

Binary 1010 1111. 0001 0110 1100
Hexadecimal A F . 1 6 C

以小数点为
分界线向左
向右划分

不够3位添0

不够4位添0

Octal – Hexadecimal Conversion

★ Convert to **Binary** as an intermediate step

Example:

(0 1 0 1 1 0 . 0 1 0)₂

内容



shutterstock.com • 1161804748

○ 2.1 数制

○ 2.2 算术运算

- 2.2.1 二进制数的算术运算
- 2.2.2 八进制数的算术运算
- 2.2.3 十六进制数的算术运算

○ 2.3 数制之间的转换

- 2.3.1 R进制转换为十进制
- 2.3.2 十进制转换为二进制
- 2.3.3 二进制数与八进制数、十六进制数之间的相互转换

○ 2.4 计算机中数的表示方法

- 2.4.1 原码及其运算
- 2.4.2 补码及其运算
- 2.4.3 反码及其运算

○ 2.5 计算机中的码

- 2.5.1 码的概念
- 2.5.2 数值编码
- 2.5.3 字符码和其它码
- 2.5.4 检错码和纠错码

○ 本章小结

计算机中数的表示方法

- 二进制码正负号需要在计算机中数值化
 - 通常0代表“+”，1代表“-”
 - 数值化的二进制数称为机器数
- 机器数通常由3种表示方法：原码、补码和反码



原码

○ 原码（true form）格式 = 符号s + 数值m



- s为符号位，0表示正数，1表示负数
- m为有效数值，也称为尾数，表示该数的绝对值
- 若符号数的位长为8，

$$\begin{aligned} N_1 &= +10010 \\ [N_1]_{\text{原}} &= 00010010 \end{aligned}$$

$$\begin{aligned} N_2 &= -10010 \\ [N_2]_{\text{原}} &= 10010010 \end{aligned}$$

- 真值为0的数有两种表示形式
- “正零” $[+0] = \overset{7}{0}\overset{6}{0}\overset{5}{0}\overset{4}{0}\overset{3}{0}\overset{2}{0}\overset{1}{0}\overset{0}{0}$
- “负零” $[-0] = \overset{7}{1}\overset{6}{0}\overset{5}{0}\overset{4}{0}\overset{3}{0}\overset{2}{0}\overset{1}{0}\overset{0}{0}$

原码的运算

- 符号位不参与运算，单独处理
- 设A、B表示绝对值

分类	运算规则	示例
同号数相加	绝对值相加，取被加（减）数的符号	$(+A) + (+B) = A + B$ $(-A) + (-B) = -(A + B)$
异号数相减		$(+A) - (-B) = A + B$ $(-A) - (+B) = -(A + B)$
同号数相减	绝对值相减，取绝对值较大值的符号	$(+A) - (+B)$ 、 $(-A) - (-B)$
异号数相加		$(+A) + (-B)$ 、 $(-A) + (+B)$

假设计算机字长5位，

$N_1 = -0011$, $N_2 = 1011$

$[N_1]_{\text{原}} = 10011$, $[N_2]_{\text{原}} = 01011$

求 $[N_1 + N_2]_{\text{原}}$ 、 $[N_1 - N_2]_{\text{原}}$

异号数相加

$$\begin{array}{r} 1011 \\ - 0011 \\ \hline 1000 \end{array}$$

$[N_1 + N_2]_{\text{原}} = 01000$

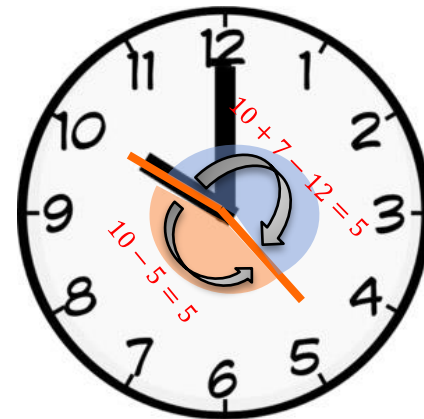
异号数相减

$$\begin{array}{r} 0011 \\ + 1011 \\ \hline 1110 \end{array}$$

$[N_1 - N_2]_{\text{原}} = 11110$

补码 (2'S COMPLEMENT) 及其运算

- 大多数计算机采用补码系统，以简化整数运算的硬件电路。
- 典型的是将减法运算变成加法运算 $A - B = A + (-B)$



原理

在某个模为 M ($\text{mod } M$) 的系统中，一个数与其模的整数倍相加减时，其值不变，即 $A = A \pm N \times M$, N 为自然数

若在模为 M 的系统中有两个数 A 和 A' ， $A + A' = M$ ， A 与 A' 互为补数。 A 为 A' 关于模 M 的补数，同样 A' 为 A 关于模 M 的补数。

利用补数的概念，可以将加法和减法统一起来：

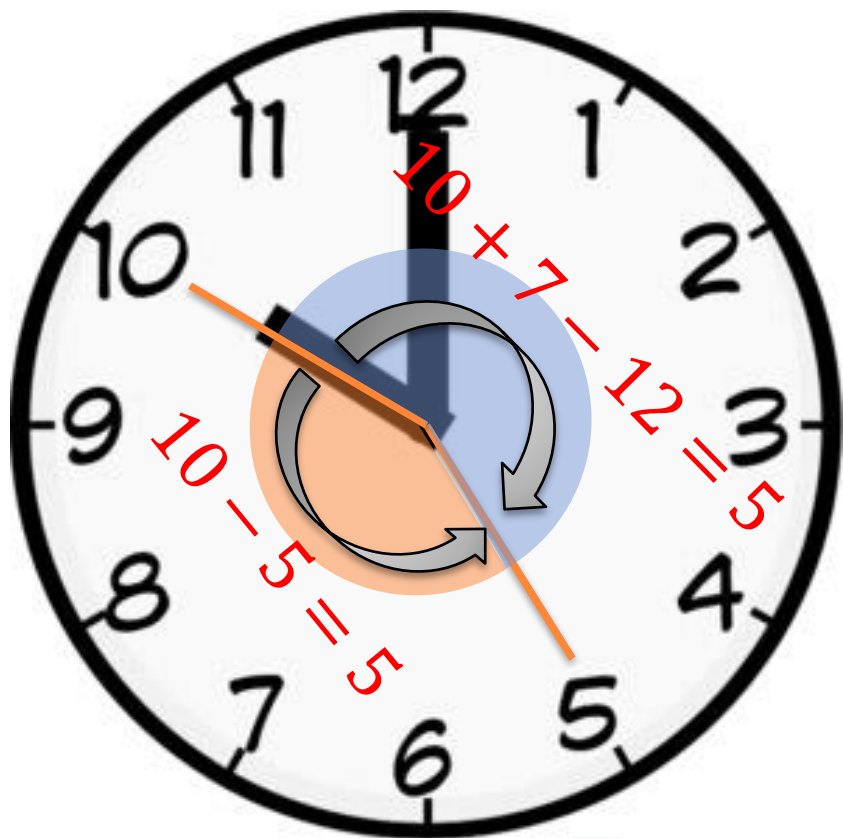
$$\begin{aligned} A - B &= A - B + M \pmod{M} \\ &= A + (M - B) \\ &= A + B' \end{aligned}$$

在模为 M 的系统中， $A - B$ 的值即为 $A + B'$ (B' 为 B 关于 M 的补数)

补数的表达是没有符号位的。
补码的最高位为符号位

在计算机中处理加、减运算，
只需要加法器和求补电路就可以了。

补码的时钟理解法



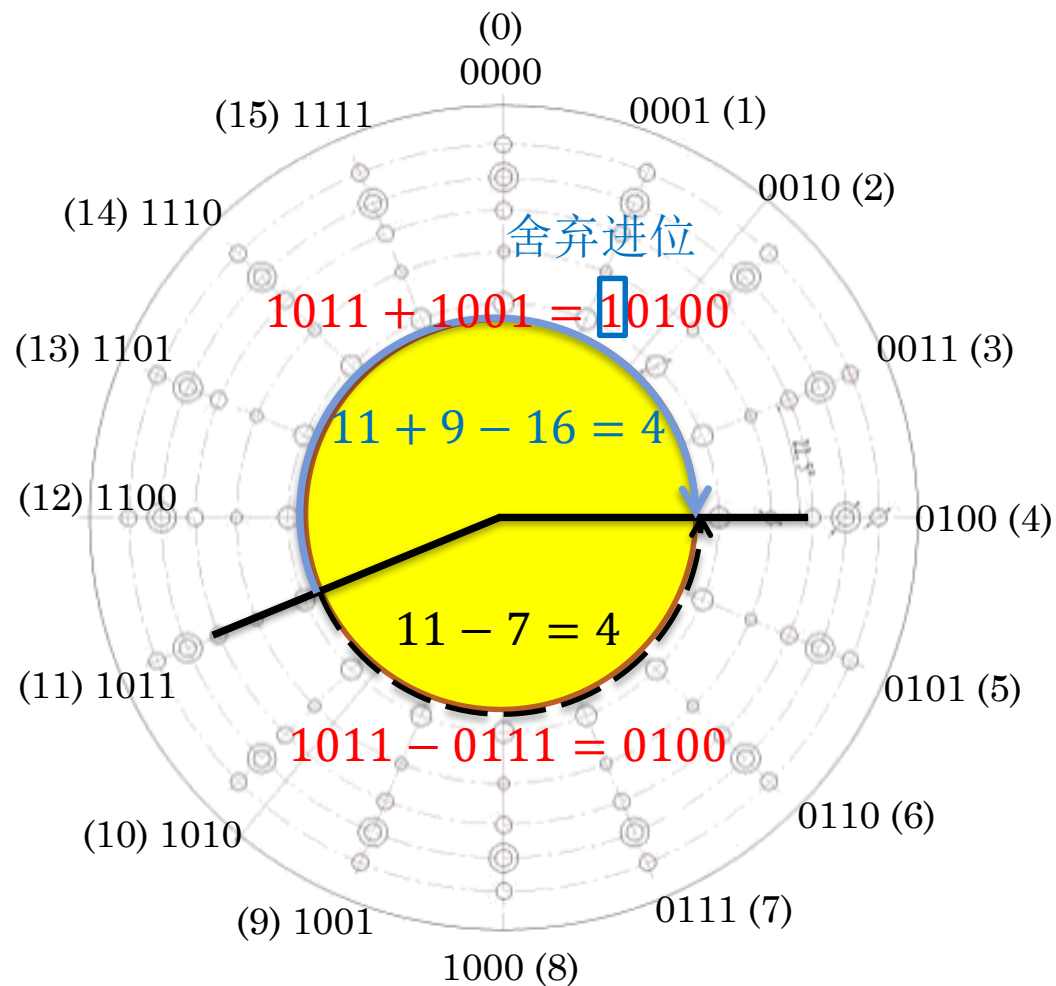
时钟的模为

$$5 + 7 = 12$$

10 - 5的减法运算可以用

10 + 7的加法运算代替

-5映射为+7，5与7对模12来说互为补数



4位二进制补码的模为

$$9 + 7 = 16$$

在舍弃进位的条件下，减去某个数可以用加上它的补码来代替

补码与模的概念

- 把一个计量单位称之为模或模数
 - 二进制运算中的模值：是计量器具的容量，或称模数。
- 在计算机中，机器数表示数据的字长（即位数）是固定的，有一定字长的限制。
 - 因此它的运算也是一种模运算
 - 当计数器计满 n 位也就是 2^n 个数后会产生溢出，又从头开始计数。
(if $n=16$, then $2^n = 65536$)
 - 其模数的大小：
 1. 对于 n 位整数（含一位符号位），则它的模数为 2^n
 2. 对于纯小数（含一位符号位），则它的模数总是2
- 二进制补码的本质，本质是用来表示负整数的

一个有符号8位的数

- 可以表示 $2^8 = 256$ 个数据,

- 最大数是 $\overset{7}{0} \overset{6}{1} \overset{5}{1} \overset{4}{1} \overset{3}{1} \overset{2}{1} \overset{1}{1} \overset{0}{1}$ (即: $+127 = 2^7 - 1 = 2^{8-1} - 1$),
- 最小数 $\overset{7}{1} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$ (-128);

- 特别注意 数值位

- $(+128) = \overset{7}{1} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0} = 2^7 = 2^{8-1};$
- 若用9位二进制表示 $(+128) = \overset{8}{0} \overset{7}{1} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$
- 通常 $(-128) = 0 - 128$
- 实际上被减数 $\overset{7}{0} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$ 需要向上一位借1, 此时被减数为 $\overset{8}{1} \overset{7}{0} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$, 即 2^8

$$[-128]_{\text{补}} = \underbrace{\overset{8}{1} \overset{7}{0} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}}_{9\text{位}} - \overset{8}{0} \overset{7}{1} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0} = \overset{7}{1} \overset{6}{0} \overset{5}{0} \overset{4}{0} \overset{3}{0} \overset{2}{0} \overset{1}{0} \overset{0}{0}$$

↑
符号位

补码及其运算

- 0 的补码只有一种形式，即000...00

考虑1位符号位

- $(N)_{comp} = \begin{cases} N & (\text{当 } N \text{ 为正数}) \\ 2^n - N & (\text{当 } -N \text{ 为负数}) \end{cases}$
 - 正数的补码与原码相同
 - 求补过程中做了减法运算：(

- n 位的计算机采用补码系统能够表达的数值范围为 N

- $-2^{n-1} \leq N \leq 2^{n-1} - 1$ 如果运算结果超出了这个范围，称为溢出。

最高位若有进位，
舍去可得正确结果

$(100 \cdots 00)_2$
↑
 n 位

$(011 \cdots 11)_2$
↑
 n 位

- 补码系统中的加法有三种数值相加的情况，设 $B, C \geq 0$,

- $[A]_{补} = [B]_{补} + [C]_{补}$

可能会溢出，符号位为 1 表示溢出

- $[A]_{补} = [B]_{补} + [-C]_{补}$

不可能溢出

- $[A]_{补} = [-B]_{补} + [-C]_{补}$

可能会溢出，符号位为 0 表示溢出



说明5位定点二进制整数的补码表数范围

- $[+x]_{max} = +1111$
 $[+1111]_{补} = 01111$
- $[+x]_{min} = +0001$
 $[+0001]_{补} = 00001$
- $[-x]_{max} = -0001$
 $[-0001]_{补} = (2^5)_{10} - (0001)_2$

$$= \overset{5}{1}\overset{4}{0}\overset{3}{0}\overset{2}{0}\overset{1}{0}\overset{0}{0} - 0001 = 11111$$
- $[-x]_{min} = -1111$
 $[-1111]_{补} = 100000 - 1111 = 10001$ 并非最小
 - 其实还有更小的负数 $[-10000]_2 = -2^4$, 其补码 $[-10000]_{补} = 100000 - 10000 = 10000$

5位整数补码系统的模为 2^5

推断

该最小数（10000）只存在于补码， -2^4 不能用原码或反码表示

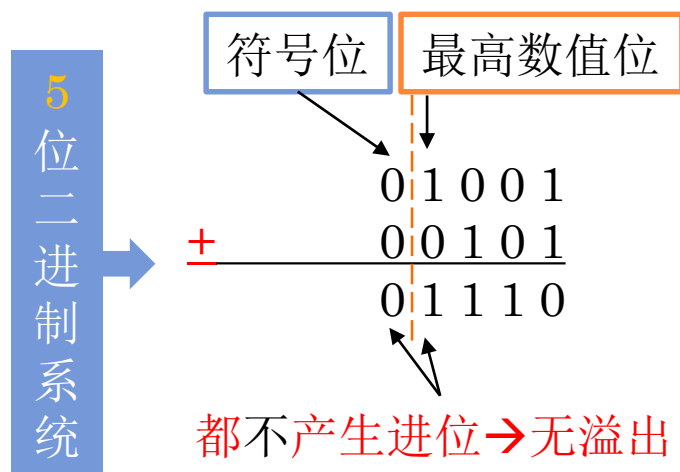
- $-10000 \leq 5\text{位定点整数} \leq +1111$, 即 $-2^4 \leq 5\text{位定点整数} \leq 2^4 - 1$
- 补码的零只有一种形式 $(+0) = +0000$, $[+0000]_{补} = 00000$
- $(-0) = -0000$, $[-0000]_{补} = 100000 - 0000 = 00000$
- 在5位二进制下, 反码只能表示0到15, -0 到 -15

补码及其运算

- 采用**单符号位法**来判断是否溢出
- 只有最高数值位与符号位“都产生进位或都不产生进位（同号）”时**不溢出**，否则溢出。
- 例如：用**补码**完成以下十进制数的加法运算

$$[A]_{\text{补}} = [B]_{\text{补}} + [C]_{\text{补}}$$

$$9 + 5 = 14 \quad \checkmark$$



	符号位的进位 C_s	最高数值位的进位 C_1
上溢	0	1
下溢	1	0

即： C_s 与 C_1 不同时溢出

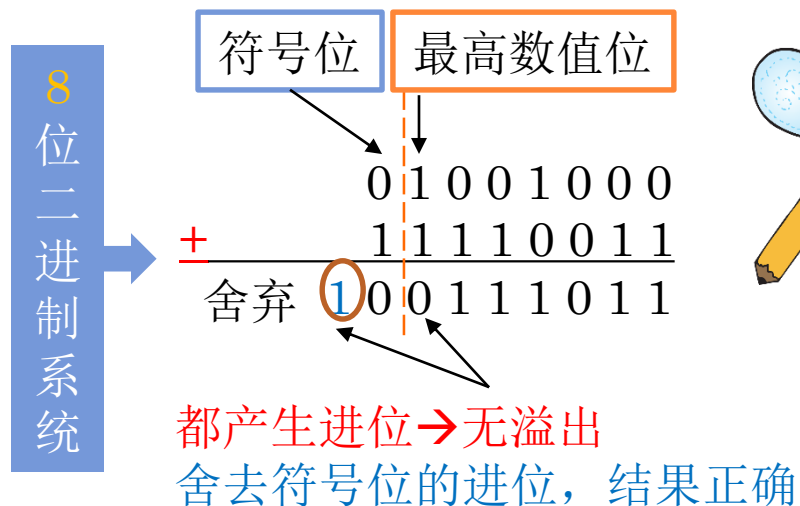
处理“不同”的逻辑符号：异或 $\oplus$

溢出逻辑判断表达式为 $V = C_s \oplus C_1$

若 $V=0$ ，表示无溢出； $V=1$ ，表示有溢出。

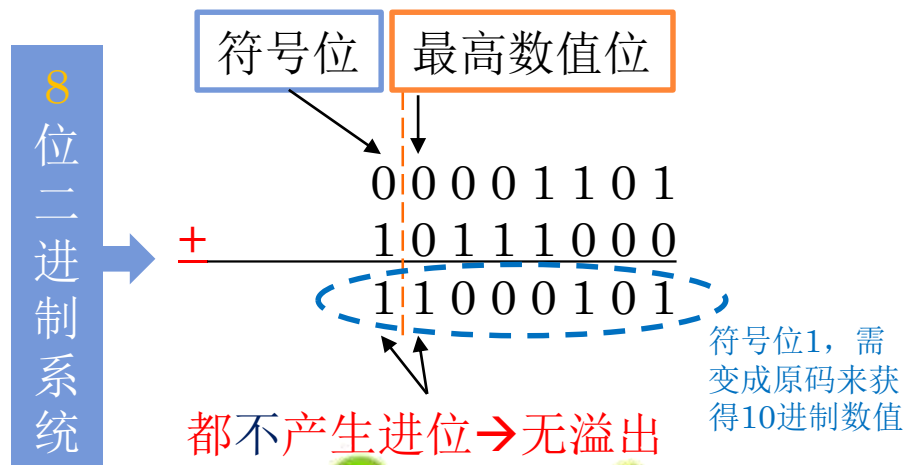
$$[A]_{\text{补}} = [B]_{\text{补}} + [-C]_{\text{补}}$$

$$72 - 13 = 59 \quad \checkmark$$



补码及其运算

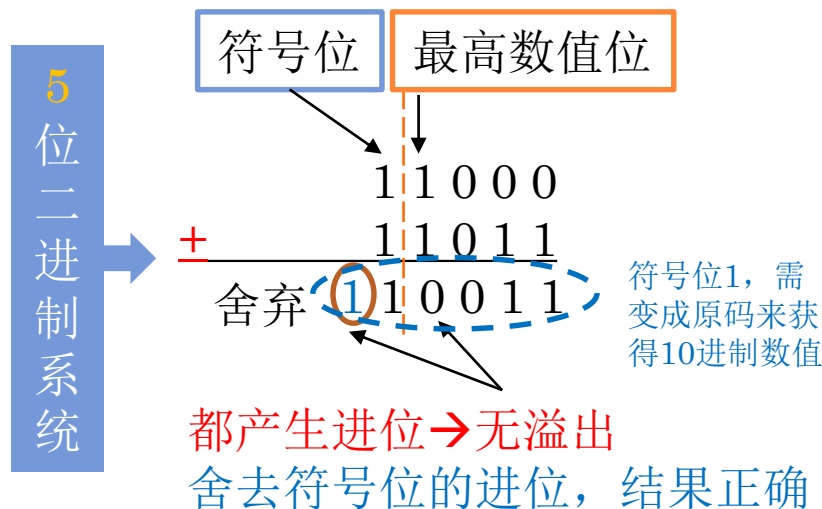
$$[A]_{\text{补}} = [B]_{\text{补}} + [-C]_{\text{补}}$$
$$13 - 72 = -59 \quad \checkmark$$



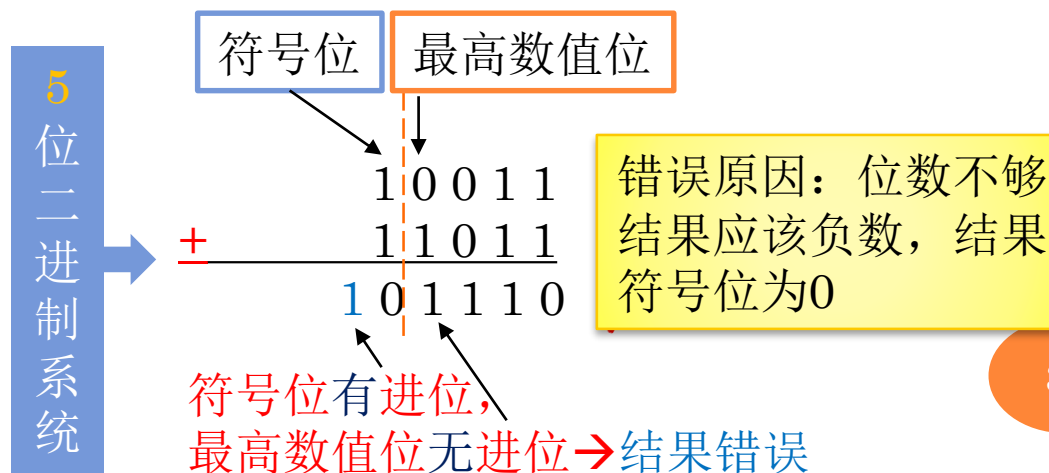
注意:

- 补码相加的和仍然为补码。
- 将两数写成补码求和时, 数值部分所取的位数必须足以表示和的最大绝对值, 否则计算结果将出错。

$$[A]_{\text{补}} = [-B]_{\text{补}} + [-C]_{\text{补}}$$
$$-8 - 5 = -13 \quad \checkmark$$



$$[A]_{\text{补}} = [-B]_{\text{补}} + [-C]_{\text{补}}$$
$$-13 - 5 =$$



反码 (1'S COMPLEMENT) 及其运算

正数的反码与原码表示相同;

$$(N)_{inv} = \begin{cases} N & (\text{当 } N \text{ 为正数}) \\ (2^n - 1) - N & (\text{当 } -N \text{ 为负数}) \end{cases}$$

n 位二进制的反码模 $2^n - 1$: 是 n 位全为 1 的二进制数, 即 $\overbrace{111 \cdots 11}^{n \text{ 位}}$

负数的反码, 符号位不变 (仍为 1), 其余各位取反。

$$N_1 = +10011$$

$$N_2 = -01010$$

$$[N_1]_{\text{反}} = 010011$$

$$[N_2]_{\text{反}} = 110101$$

真值 0 的反码有 2 种

$$[+0]_{\text{反}} = 000 \cdots 000,$$

$$[-0]_{\text{反}} = 111 \cdots 111,$$

反码的运算规则

$$[N_1 + N_2]_{\text{反}} = [N_1]_{\text{反}} + [N_2]_{\text{反}}$$

$$[N_1 - N_2]_{\text{反}} = [N_1]_{\text{反}} + [-N_2]_{\text{反}}$$

$$(N)_{\text{comp}} = (N)_{\text{inv}} + 1$$

$$[N]_{\text{补}} = [N]_{\text{反}} + 1$$



考虑1位符号位

反码 (1'S COMPLEMENT) 及其运算

Attention Please!



- 用反码求得的运算结果，如果符号位向更高位有进位，则运算结果需要再加1。

-3

11

和补码的运算不一样哦

- 例: $N_1 = -0011$, $N_2 = 1011$, 求 $[N_1 + N_2]_{\text{反}}$ 和 $[N_1 - N_2]_{\text{反}}$

解: $[N_1]_{\text{反}} = 11100$, $[N_2]_{\text{反}} = 01011$, $[-N_2]_{\text{反}} = 10100$

$$[N_1 + N_2]_{\text{反}} = 11100 + 01011$$

	符号位	最高数值位	
	1	1 1 0 0	符号位有进位, 因此还要加1
+	0	1 0 1 1	
	1	0 0 1 1 1	
+		1	
	0	1 0 0 0	表明是正数

正数的原码和反码一样
真值为: $N_1 + N_2 = 1000$

$$[N_1 - N_2]_{\text{反}} = 11100 + 10100$$

	符号位	最高数值位	
	1	1 1 0 0	符号位有进位, 因此还要加1
+	1	0 1 0 0	
	1	1 0 0 0 0	
+		1	
	1	0 0 0 1	表明是负数的反码

真值为: $N_1 - N_2 = -1110$

十进制与二进制转换总结

负十进制数如何转换成二进制

	Decimal 十进制	Signed Binary 原码	Radix minus one complement反码	radix complement 补码 (RC)
Positive	+45	00101101	00101101	00101101
Negative	-45	10101101	11010010	11010011
Binary Sum		11011010	11111111	00000000
Decimal Sum	0	-90	-127	0

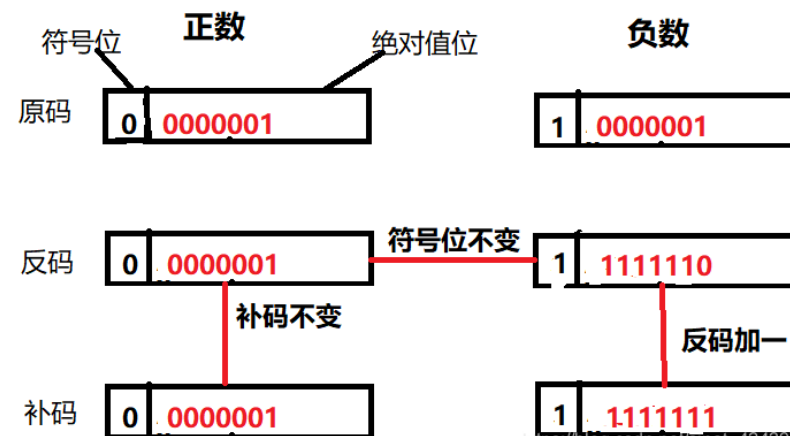


二进制中的原码、反码、补码

对于有符号数而言：

- (1) 二进制的最高位是符号位：0表示正数，1表示负数
- (2) 正数的原码、反码、补码都一样；
- (3) 负数的反码 = 它的原码符号位不变，其它位取反（0 → 1；1 → 0）
- (4) 负数的补码 = 它的反码 + 1；（补码转换为原码，用逆推法即可）
- (5) 将负数的补码再求补得到原码，即“补码的补码等于原码”；
- (6) 在计算机运算的时候，都是以补码的方式来运算的；

NOTE: 如果想将 二进制 转为 十进制，必须使用 二进制的原码



内容



shutterstock.com • 1161804748

- 2.1 数制
- 2.2 算术运算
 - 2.2.1 二进制数的算术运算
 - 2.2.2 八进制数的算术运算
 - 2.2.3 十六进制数的算术运算
- 2.3 数制之间的转换
 - 2.3.1 R进制转换为十进制
 - 2.3.2 十进制转换为二进制
 - 2.3.3 二进制数与八进制数、十六进制数之间的相互转换
- 2.4 计算机中数的表示方法
 - 2.4.1 原码及其运算
 - 2.4.2 补码及其运算
 - 2.4.3 反码及其运算
- 2.5 计算机中的码
 - 2.5.1 码的概念
 - 2.5.2 数值编码
 - 2.5.3 字符码和其它码
 - 2.5.4 检错码和纠错码
- 本章小结

计算机中的码

○ 码的概念

- 码是一种用来表达信息的方法，这种方法有一定的规则。在日常生活中随处可见码的应用，比如你的学号

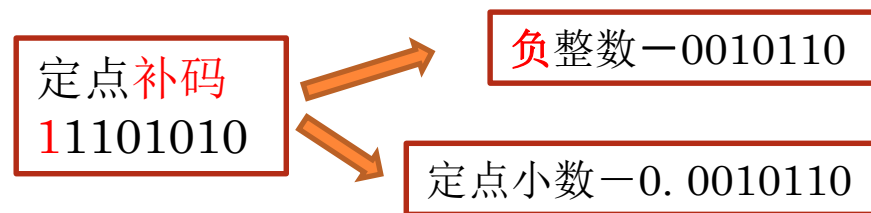
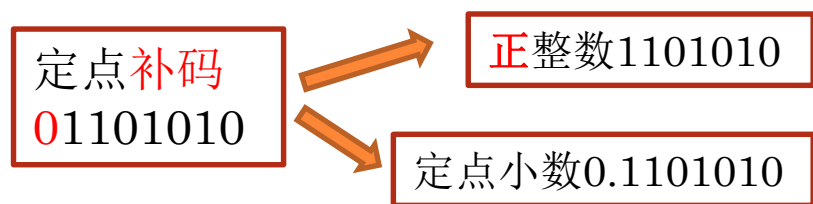


动态二维码

数值编码

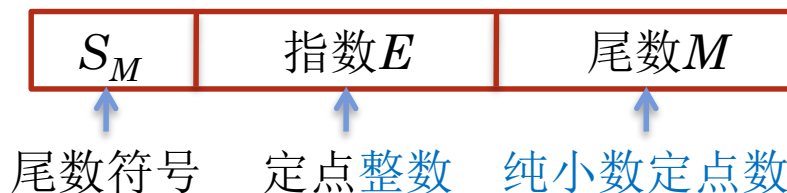
处理或者存储数值的编码

- 按照小数点位置的固定与否分为
 - 定点数**（**小数点**可固定在任何位置）
 - 通常固定在数值部分的**最低位之后**（**纯整数**）
 - 通常固定在数值部分的**最高位之前**（**纯小数**）



- 浮点数**（数的科学表示法）
 - 常用于**大型计算机**中（不多讨论）
 - 包含阶符、阶码、数符和尾数

$$N = M \times 2^E$$



字符码和其它码

常用的BCD码

格雷码（无权码） 有权码 余3码（无权码）

Decimal	Binary	Gray	NBCD 8421	Excess-3
0	0	0000	0000	0011
1	1	0001	0001	0100
2	10	0011	0010	0101
3	11	0010	0011	0110
4	100	0110	0100	0111
5	101	0111	0101	1000
6	110	0101	0110	1001
7	111	0100	0111	1010
8	1000	1100	1000	1011
9	1001	1101	1001	1100



常用的编码

- 字母字符编码
- 数字字符编码

BCD码（Binary-Coded Decimal）

- 十进制数码（0 ~ 9）共有十个，至少用4位二进制码表示。
 $2^3 < 10 < 2^4$
- 原则上可以对每个十进制数码任意指定一组二进制代码。

NOTE: BCD码不存在“逢二进一”的进位关系，不能按二进制运算法则进行运算。

- 相对于一般的浮点式记数法，采用BCD码，既可保存数值的精确度，又可免去使计算机作浮点运算时所耗费的时间。此外，对于其它需要高精确度的计算，BCD编码亦很常用。

1111	Bit-Nr.	4	3	2	1	NBCD + 0011
1110	Wertigkeit	2^3	2^2	2^1	2^0	
1010	Dezimalziffern	0				
1011		1				
1001		2				
1000		3				
		4				
		5				
		6				
		7				
		8				
		9				

几种常用的BCD码

	有权码	有权码	有权码	无权码	无权码
十进制数	8421码	5421码	2421码	余3码	BCD格雷码
0	0000	0000	0000	0011	0000
1	0001	0001	0001	0100	0001
2	0010	0010	0010	0101	0011
3	0011	0011	0011	0110	0010
4	0100	0100	0100	0111	0110
5	0101	1000	1011	1000	0111
6	0110	1001	1100	1001	0101
7	0111	1010	1101	1010	0100
8	1000	1011	1110	1011	1100
9	1001	1100	1111	1100	1000

8421码：恒权码

- 有权BCD码：4位二进制数中每一位数码都有确定的位权值。

5421码在特定的计数器应用中具有优势，因为其高位连续5个0后连续5个1的特性，使得计数器可以产生对称方波输出。

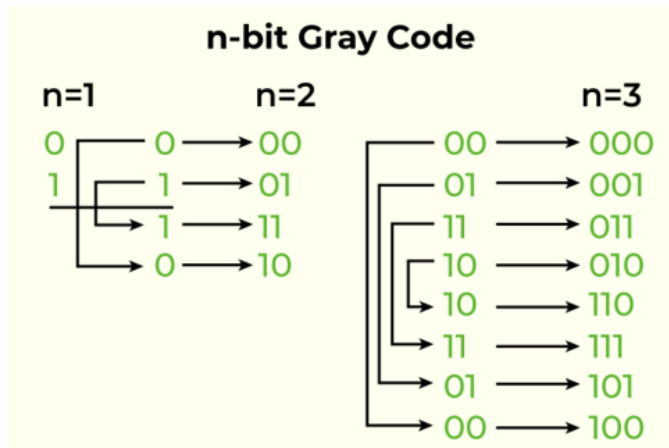
- 这种特性使得5421码在某些特定的数字电路和计算机系统有着广泛的应用。

2421码：对9的自补代码

- 即每一个2421码只要与自身按位取反 ($D = 9 - d$)

在一组数的编码中，若任意两个相邻的代码只有一位二进制数不同，则成为格雷码（Gray Code）

- 由于最大数与最小数之间也仅有一位数不同，首尾相连，又称为“循环码”。
- 常用于计数器中，以防止多计数或者少计数。



Number				
0	0	0	0	0
0	0	0	0	1
0	0	0	1	0
0	0	0	1	1
0	0	1	0	0
0	0	1	0	1
0	0	1	1	0
0	0	1	1	1

Gray				
0	0	0	0	0
0	0	0	0	1
0	0	0	1	1
0	0	0	1	0
0	0	1	1	0
0	0	1	1	1
0	0	1	0	1
0	0	1	0	0

Vertical dimensions: B₁ (rows 1-2), B₂ (rows 2-3), B₃ (rows 3-8)

Number				
0	0	0	0	0
0	0	0	0	1
0	0	0	1	0
0	0	0	1	1
0	0	1	0	0
0	0	1	0	1
0	0	1	1	0
0	0	1	1	1
0	1	0	0	0
0	1	0	0	1
0	1	0	1	0
0	1	0	1	1
0	1	1	0	0
0	1	1	0	1
0	1	1	1	0
0	1	1	1	1
1	0	0	0	0
1	0	0	0	1
1	0	0	1	0
1	0	0	1	1
1	0	1	0	0
1	0	1	0	1
1	0	1	1	0
1	0	1	1	1
1	1	0	0	0
1	1	0	0	1
1	1	0	1	0
1	1	0	1	1
1	1	1	0	0
1	1	1	0	1
1	1	1	1	0
1	1	1	1	1

Gray				
0	0	0	0	0
0	0	0	0	1
0	0	0	1	1
0	0	0	1	0
0	0	1	1	0
0	0	1	1	1
0	0	1	0	1
0	0	1	0	0
0	1	1	0	0
0	1	1	0	1
0	1	1	1	0
0	1	1	1	1
0	1	0	1	0
0	1	0	1	1
0	1	0	0	0
1	1	0	0	0
1	1	0	0	1
1	1	0	1	1
1	1	0	1	0
1	1	1	1	0
1	1	1	1	1
1	1	1	0	1
1	1	1	0	0
1	0	1	0	0
1	0	1	0	1
1	0	1	1	1
1	0	0	1	0
1	0	0	1	1
1	0	0	0	1
1	0	0	0	0

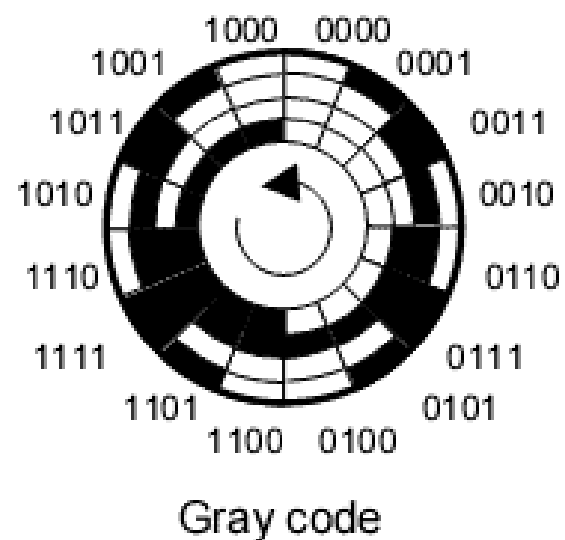
Vertical dimensions: B₁ (rows 1-2), B₂ (rows 2-3), B₃ (rows 3-8), B₄ (rows 8-16), B₅ (rows 16-32)

Number				
0	0	0	0	0
0	0	0	0	1
0	0	0	1	0
0	0	0	1	1
0	0	1	0	0
0	0	1	0	1
0	0	1	1	0
0	0	1	1	1
0	1	0	0	0
0	1	0	0	1
0	1	0	1	0
0	1	0	1	1
0	1	1	0	0
0	1	1	0	1
0	1	1	1	0
0	1	1	1	1
0	1	1	0	0
0	1	1	0	1
0	1	1	1	0
0	1	1	1	1

Gray				
0	0	0	0	0
0	0	0	0	1
0	0	0	1	1
0	0	0	1	0
0	0	1	1	0
0	0	1	1	1
0	0	1	0	1
0	0	1	0	0
0	1	1	0	0
0	1	1	0	1
0	1	1	1	1
0	1	1	1	0
0	1	0	1	0
0	1	0	1	1
0	1	0	0	1
0	1	0	0	0
1	1	0	0	0
1	1	0	0	1
1	1	0	1	0
1	1	0	1	1
1	1	1	0	0
1	1	1	0	1
1	1	1	1	0
1	1	1	1	1

Vertical dimensions: B₁ (rows 1-2), B₂ (rows 2-3), B₃ (rows 3-8), B₄ (rows 8-16)

GRAY CODE (循环码)



8421BCD (NATURAL BCD) 码形式上看与普通二进制码没有区别，但是不能按二进制“逢二进一”进行运算

○ 用8421BCD码表示 $(365)_{10}$

十进制	3	6	5	
	↓	↓	↓	
8421BCD码	0011	0110	0101	连起来得 $(0011\ 0110\ 0101)_{NBCD}$

- 两个BCD码相加时，
 - 每一位的大小不能超过9（因为BCD码只表示0~9的十进制数字）
 - 考虑低位可能有进位，全加和不能超过 $9+9+1=19$
- 二进制表示的0~19为 $(00000)_2 \sim (10011)_2$
- NBCD码应为 $(00000)_{NBCD} \sim (11001)_{NBCD}$
 $(1\ 9)_{10}$

BCD加法中的
调整值为0110

- 当相加后某一位大于9（即1001）时，出现NBCD值错误。
 - 需要将结果再加上6（即0110），可调整为正确的NBCD值。

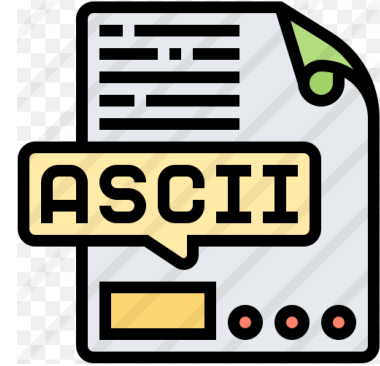
例如：用3位8421BCD 码计算两个十进制数加法：
448 + 489

按照NBCD加法，

- 最低位 $8+9=17$ ，即 $1000 + 1001 = (10001)_2$
- 调整结果再加6，即 $10001 + 0110 = (10111)_{NBCD}$
 $(1\ 7)_{10}$
- 次低位 $4+8=12$ ，即 $0100 + 1000 = (1100)_2$
- 调整结果再加6，即 $1100 + 0110 = (10010)_{NBCD}$
- 加上低位向本位的进位得 $(10011)_{NBCD}$
 $(1\ 2)_{10}$
- 加上低位向本位的进位得 $(10011)_{NBCD}$
 $(1\ 3)_{10}$
- 最高位 $4+4=8$ ，即 $0100 + 0100 = (1000)_2$
- 结果 $8 < 9$ ，不用调整结果
- 加上低位向本位的进位得 $(1001)_{NBCD}$
- 最终结果为 $(1001\ 0011\ 0111)_{NBCD}$ ，即937

	1 1	← 进位
	4 4 8	
+	4 8 9	
	9 3 7	

ASCII ((AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE):



- 美国信息交换标准代码（ASCII）是基于拉丁字母的一套电脑编码系统，**主要用于显示现代英语和其他西欧语言。**
 - 最通用的信息交换标准
 - 一个ASCII码由8位二进制码组成，其**最高位(b_7 初始为0)**用作**奇偶校验位**
 - 到目前为止**标准/基础**ASCII码共定义了**128个字符**
- 在计算机中，**所有的数据**在存储和运算时都要使用**二进制数表示**
 - 因为计算机用高电平和低电平分别表示1和0

ASCII 字符代码表 一

高四位 低四位		ASCII非打印控制字符										ASCII 打印字符													
		0000					0001					0010		0011		0100		0101		0110		0111			
		0					1					2		3		4		5		6		7			
		+进制	字符	ctrl	代码	字符解释	+进制	字符	ctrl	代码	字符解释	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符	+进制	字符
0000	0	0	BLANK NULL	^@	NUL	空	16	▶	^P	DLE	数据链路转意	32		48	0	64	@	80	P	96	`	112	p		
0001	1	1	☺	^A	SOH	头标开始	17	◀	^Q	DC1	设备控制 1	33	!	49	1	65	A	81	Q	97	a	113	q		
0010	2	2	☺	^B	STX	正文开始	18	↕	^R	DC2	设备控制 2	34	"	50	2	66	B	82	R	98	b	114	r		
0011	3	3	♥	^C	ETX	正文结束	19	!!	^S	DC3	设备控制 3	35	#	51	3	67	C	83	S	99	c	115	s		
0100	4	4	◆	^D	EOT	传输结束	20	¶	^T	DC4	设备控制 4	36	\$	52	4	68	D	84	T	100	d	116	t		
0101	5	5	♣	^E	ENQ	查询	21	♫	^U	NAK	反确认	37	%	53	5	69	E	85	U	101	e	117	u		
0110	6	6	♠	^F	ACK	确认	22	■	^V	SYN	同步空闲	38	&	54	6	70	F	86	V	102	f	118	v		
0111	7	7	●	^G	BEL	震铃	23	↑↓	^W	ETB	传输块结束	39	'	55	7	71	G	87	w	103	g	119	w		
1000	8	8	◼	^H	BS	退格	24	↑	^X	CAN	取消	40	(	56	8	72	H	88	X	104	h	120	x		
1001	9	9	○	^I	TAB	水平制表符	25	↓	^Y	EM	媒体结束	41	)	57	9	73	I	89	Y	105	i	121	y		
1010	A	10	◻	^J	LF	换行/新行	26	→	^Z	SUB	替换	42	*	58	:	74	J	90	Z	106	j	122	z		
1011	B	11	♂	^K	VT	竖直制表符	27	←	^[	ESC	转意	43	+	59	;	75	K	91	[	107	k	123	{		
1100	C	12	♀	^L	FF	换页/新页	28	└	^\ FS	文件分隔符	44	,	60	<	76	L	92	\	108	l	124				
1101	D	13	♪	^M	CR	回车	29	↔	^] GS	组分隔符	45	-	61	=	77	M	93	]	109	m	125	}			
1110	E	14	🎵	^N	SO	移出	30	▲	^6 RS	记录分隔符	46	.	62	>	78	N	94	^	110	n	126	~			
1111	F	15	🕒	^O	SI	移入	31	▼	^- US	单元分隔符	47	/	63	?	79	O	95	_	111	o	127	Δ	^Back space		

注：表中的ASCII字符可以用:ALT + “小键盘上的数字键” 输入

常见ASCII码的大小规则

- $0 \sim 9 < A \sim Z < a \sim z$.
 1. 数字比字母要小。如 “7” < “F”;
 2. 数字0比数字9要小，并按0到9顺序递增。如 “3” < “8”;
 3. 字母A比字母Z要小，并按A到Z顺序递增。如 “A” < “Z”;
 4. 同个字母的大写字母比小写字母要小32。如 “A” < “a”。
- 几个常见字母的ASCII码大小：
 - “A”为 65;
 - “a”为 97;
 - “0”为 48;
 - 空格为32.

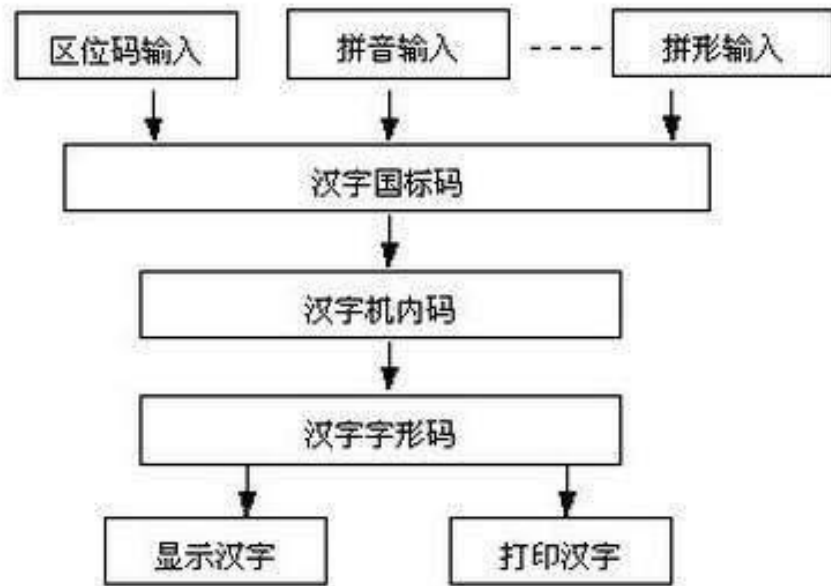


ASCII码的问题

- 在英语中，用128个符号编码便可以表示所有，但是用来表示其他语言，128个符号是不够的。
 - 一些欧洲国家就决定，利用字节中闲置的最高位编入新的符号。这些欧洲国家使用的编码体系，可以表示最多256个符号。
- 不同的国家有不同的字母，代表的字母却不一样。
 - 0--127表示的符号是一样的，不一样的只是128--255的这一段。
- 亚洲国家的文字，使用的符号就更多了，汉字就多达10万左右。
 - 一个字节只能表示256种符号，肯定是不够的，就必须使用多个字节表达一个符号。
 - 比如，简体中文常见的编码方式是 GB2312，使用两个字节表示一个汉字，理论上最多可以表示 $256 \times 256 = 65536$ 个符号

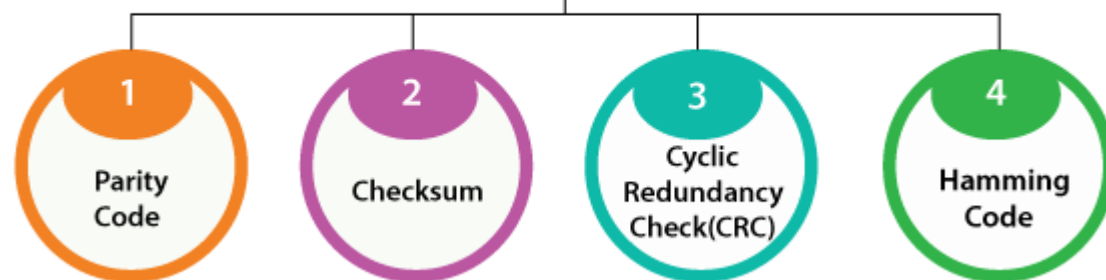
汉字编码

- GB2312、GBK、GB18030 是汉字字符编码方案的国家标准。



- 《信息交换用汉字编码字符集基本集》（GB2312-80）,即国标码
 - 规定了汉字信息交换的基本图形字符及其二进制编码
 - 适用于一般汉字处理、汉字通信等系统之间的信息交换
 - 采用两个字节表示 1 个汉字。
 - 共收录 7445 个字符，其中汉字 6763 个。

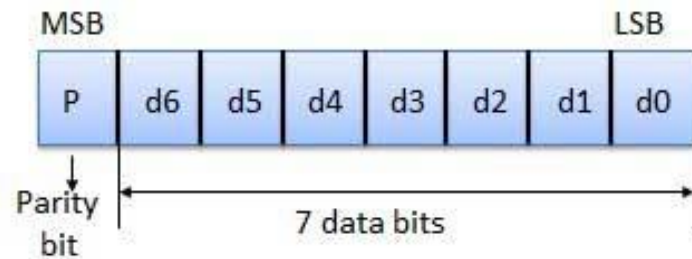
检错码和纠错码



- **检错码**：为了能够判别数据在传输过程中是否发生了错误，需要对传输的数据检错。
- **纠错码**：对数据传送中出现错误的数据进行纠正。
- 传输的数据通常是二进制数据，错误可能出现在其中的一位或者几位。引起错误的原因有多种：
 - 硬件错误
 - 干扰（噪声）： single event upset (SEU) 单粒子翻转
 - 一些突发事件
- 若 I, J 分别代表 n 位的二进制信息码
 - 码重 $w(I)$ ：定义为 I 中的二进制信息位为 1 的个数
 - 码距 $d(I, J)$ ：等于 I 与 J 对应的信息位不相同的个数

$$\begin{array}{rcl}
 I & = & 1\ 1\ 1\ 0\ 1\ 1\ 0\ 0 \\
 J & = & 1\ 1\ 0\ 0\ 0\ 1\ 0\ 0
 \end{array}$$

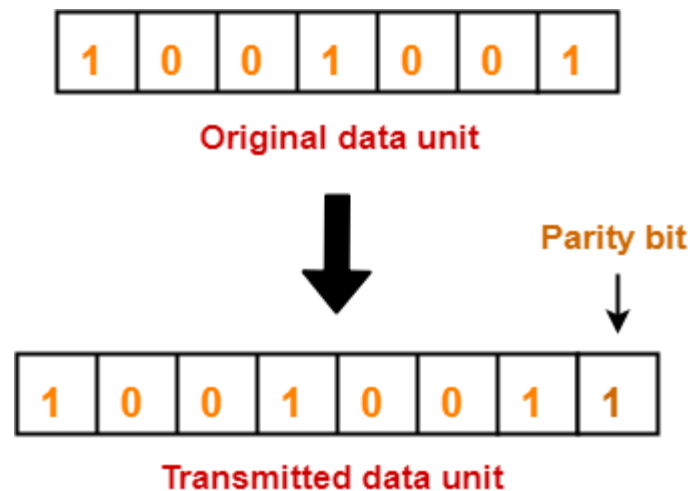
- $w(I) = 5$
- $w(J) = 3$
- $d(I, J) = 2$



奇偶校验码

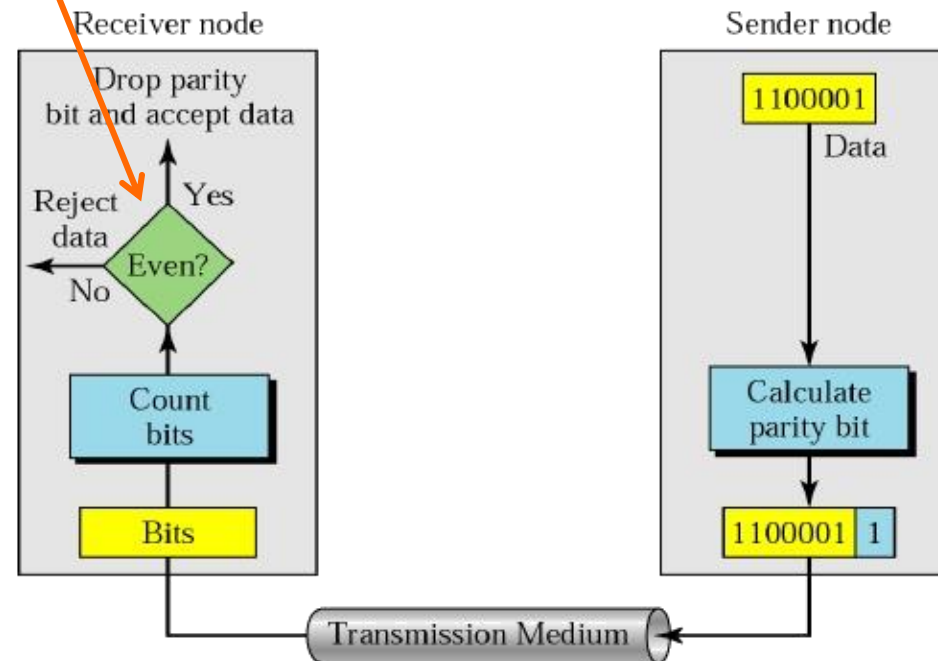
- 奇偶校验码由信息位和校验位（冗余部分）两部分组成。
 - 根据接收到的信息中1的奇偶性，发现奇偶校验码在传输中是否出现奇数个位的错误。

- 该方法简单易行，实现代价低。
 - 出现一位数码错误的概率非常大。
 - 但是，不能发现偶数位的错误
 - 奇偶校验也无法知道具体哪一位错了。
 - 它是使用一位数据能够达到的最好的校验码



	偶校验	奇校验
Original Data	Even Parity	Odd Parity
0 0 0 0 0 0 0	0	1
0 1 0 1 1 0 1 1	1	0
0 1 0 1 0 1 0 1	0	1
1 1 1 1 1 1 1 1	0	1
1 0 0 0 0 0 0 0	1	0
0 1 0 0 1 0 0 1	1	0

(1) Parity Checking:



五位代码中包含两个1的十进制编码

- 具有检错功能
 - 5位码中2位值为1，3位值为0。
 - 若验证发现5位中不是2位为1，则表示有错。

Decimal	5-bit Code	Decimal	5-bit Code
0	00011	5	01010
1	00101	6	10010
2	01001	7	01100
3	10001	8	10100
4	00110	9	11000

$$C_5^2 = 10 \rightarrow \text{五位代码}$$

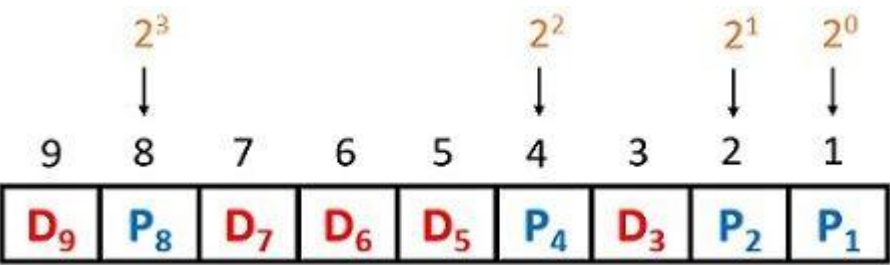
海明码 (HAMMING CODE)

- 由于海明编码简单，它们被广泛应用于内存 (RAM)
- 是简单奇偶校验码的扩展，采用了多个奇偶校验位或监督位
- 属于**纠错码**：使用多余的校验位来识别1位错误
 - 1类海明码：有1位纠错功能，无2位错误检错功能
 - 2类海明码：有1位纠错功能，有2位错误检错功能
 - SECDED (single error correction, double error detection)

Bit position	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Encoded data bits	p1	p2	d1	p4	d2	d3	d4	p8	d5	d6	d7	d8	d9	d10	d11	p16	d12	d13	d14	d15
2ⁿ	p1	X		X		X		X		X		X		X		X		X		X
Parity bit coverage	p2		X	X			X	X			X	X			X	X			X	X
	p4				X	X	X	X					X	X	X	X				X
	p8								X	X	X	X	X	X	X					
	p16																X	X	X	X

The **source** encodes the message by inserting redundant bits within the message.
 The **destination** decodes this message and find the bit position that has error.

EXAMPLE OF HAMMING CODE



7	6	5	4	3	2	1	How to calculate parity bits
D_7	D_6	D_5	P_4	D_3	P_2	P_1	This represents the full codeword
D_7	D_6	D_5	P_4				P_4 - Even parity of $D_7 D_6 D_5$
D_7	D_6			D_3	P_2		P_2 - Even parity of $D_7 D_6 D_3$
D_7		D_5		D_3		P_1	P_1 - Even parity of $D_7 D_5 D_3$

Data: 1001101 Code: 10011100101

Hamming Code

Example:



本章小结

- 数字电路和计算中，信息是采用二进制表示的。
 - 数字信息通常用二进制数来表示。为了书写方便，有时会用到八进制和十六进制。
 - 非数字信息也用二进制数来表示。
- 二进制两个数码0和1用电路方便实现。
 - 电压的高低、电流的有无、电子开关的通断
- 数字和数码两个概念
- 计算机中机器数的三种形式：原码、反码、补码。
 - 在算术运算中常采用补码的形式。
- 在信息传输中为了检错和纠错，需要采用检错码和纠错码。
- 应会内容：原码、反码、补码，P35 + P40 + P41



教材勘误表

页码	位置	错漏
30	2.A=B-C情况下面第3行	[C]补 漏写负号，补码运算中的数都是补码形式
36	表2.10上方文字	ASCII码序数很多错误
48	2.17题	应写为“原码相减的结果”，要求变成补码后运算
48	2.20题	应写为“对以下二进制数用反码完成运算”，而不是“两个反码相减”，容易引起歧义
48	2.24题第（4）小题	最高位0001没有相应的余3码